Correlation and Trace IDs

one ID, carried everywhere

Watch the 4-min video

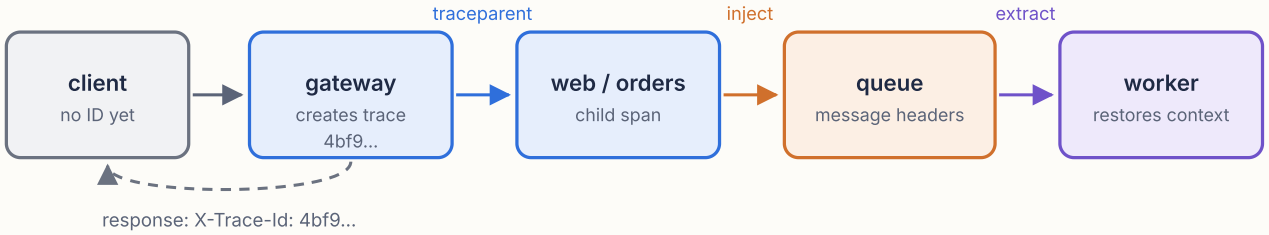
In one line: create one trace ID at the edge, propagate it on every HTTP/gRPC call, queue message and background job (the W3C `traceparent` header), and write it into every log line; then one search shows a request's whole journey, in order.

1 Which lines belong to that payment?

A customer reports a failed payment at 10:42. Five services write thousands of lines a second. Without a shared ID you grep by timestamp and hope, and the lines from concurrent requests interleave. One ID, carried everywhere, turns this into a single query.

2 Correlation ID, request ID, trace ID

A **correlation ID** (or request ID) is a unique value created where a request enters your system. Every service passes it on and every log line includes it. Today the standard carrier is the W3C **traceparent** header, which holds the **trace ID** for the whole request, the parent span ID and a sampled flag (lesson 63). OpenTelemetry creates and forwards it automatically on HTTP and gRPC calls (lesson 64). If you also have a legacy request ID, log both; return the trace ID to the client in a response header.




```
ONE SEARCH: TRACE_ID=4BF9...
10:42:01 web trace=4bf9 span=a1 POST /orders 202
10:42:03 worker trace=4bf9 span=c7 send_invoice start
10:42:04 worker trace=4bf9 span=c7 SMTP timeout · retry 1
10:42:09 worker trace=4bf9 span=d2 invoice sent
```

Figure 1. The trace ID created at the gateway follows the request through HTTP, a queue and a worker, and lands on every log line.

3 Linking logs to traces

Configure the logger to add `trace_id` and `span_id` from the current context to every structured line (lesson 60). Then you can jump from a slow span to its exact log lines, or from an error log to the full trace, in tools such as Grafana (Loki ↔ Tempo) or Kibana ↔ APM. No more matching timestamps by eye.

```
class TraceContextFilter(logging.Filter):
    def filter(self, record):
        ctx = trace.get_current_span().get_span_context()
        record.trace_id = format(ctx.trace_id, "032x")   # 32 hex chars, as in traceparent
        record.span_id = format(ctx.span_id, "016x")
        return True
# or: LoggingInstrumentor().instrument(set_logging_format=True)
```

Figure 2. A logging filter that copies the current trace and span IDs onto each record; OTel's logging instrumentation can do the same for you.

4 Async boundaries: where the trail goes cold



Figure 3. HTTP propagation is automatic once instrumented. These three need explicit care, or the instrumentation for your queue library.

5 Rules for IDs

Rule	Why
Validate client-supplied IDs (format, length) or start a new trace and **link** the old one	untrusted input can inject garbage into logs or hijack sampling
Random and meaningless (128-bit trace ID)	no user data or sequence numbers to leak or guess
Show it to users on error pages and in API error bodies	support asks for one ID and hands it straight to engineering
Same field name everywhere (`trace_id`)	one query works across all services

WORKED EXAMPLE: A CELERY TASK KEEPS THE TRACE

The web request has trace ID `4bf9…`. It enqueues `send_invoice`. The OpenTelemetry Celery instrumentation writes `traceparent` into the task's message headers on publish; the worker extracts it and starts the task span as a child. A logging filter (Figure 2) adds the trace ID to every record, so the worker's lines carry the same ID as the web request. One search for `4bf9` shows the request, the task, its SMTP timeout, the retry and the success, in order (Figure 1).

COMMON MISTAKES

- A new ID generated in every service instead of propagating the incoming one.
- Forgetting the queue: the consumer starts a fresh trace and the story splits in two.
- Putting the trace ID in a **metric label** or a Loki label: unbounded cardinality (lessons 59, 66).
- Trusting any `X-Request-ID` from the internet without validation.

6 Key terms

Correlation ID: one ID shared by everything a request causes

Trace ID: 128-bit ID of the whole trace (32 hex chars)

traceparent: W3C header carrying trace ID, parent span ID, flags

Inject / extract: write / read context into carrier headers

Span link: reference to a related trace without a parent relationship

Context: the current span, carried per thread or task

7 Interview questions

Q1. How do you follow one request through logs from five services? BASIC

Generate a trace ID at the edge, propagate it with the W3C traceparent header across every call, queue and job, and include trace_id (and span_id) in every structured log line; one search then shows the whole journey.

Q2. How do you keep the trail across a message queue? BASIC

Inject the trace context into the message headers when publishing and extract it in the consumer before processing, usually via the queue library's OTel instrumentation.

Q3. Support says 'the customer got an error'. How do you make that actionable? SCENARIO

Show the trace ID on error pages and in API error responses; support collects it and engineering searches logs and traces by it directly.

Q4. A nightly batch job processes 10,000 orders. One trace or many? SCENARIO

Usually one trace per job run (or per item for long jobs), with span links back to the triggering request or schedule, so traces stay a manageable size.

Q5. Should you accept an incoming traceparent from a public client? TRICKY

Accept only well-formed values, and many systems start a new trace at the edge and link the client's one, so outsiders can't force sampling or pollute your traces.

Q6. Why not just use timestamps to correlate logs? TRICKY

Concurrent requests interleave and clocks skew between hosts; only a shared ID reliably groups lines from one request.

Next: SD-N-008 · Log Aggregation backendroadmap.com